TrekTrail

Trek Management App

Project Report

Modern Application Development I (MAD-I)

MAY 2026 Term

IIT Madras BS Program

Student: ***NAYAKANTI AMRUTH RAJ***

Roll Number: *23f1000530*

1. Project Overview

Problem Statement

Trekking expedition management is traditionally handled through manual coordination between organizers, on-ground staff, and trekkers, which makes it difficult to control booking integrity, track slot availability, and maintain a reliable history of past expeditions. There is a need for a centralized digital platform that manages the entire lifecycle of a trek — from publishing and staffing to booking and completion — while enforcing strict business rules around availability and booking validity.

Solution

The Trek Management App is a full-stack, multi-user web application that connects three key stakeholders — Admins, Trek Staff, and Trekkers — through a unified platform. Admins publish and manage treks through a controlled lifecycle. Trek Staff manage the treks assigned to them, updating slots, status, and participant details. Trekkers can discover, search, and book treks, and maintain a complete trekking history. The system enforces server-side booking integrity rules, including prevention of overbooking and duplicate bookings, and only allows bookings while a trek is in the Open state.

Tech Stack

Component	Technology
Backend	Python Flask (Application Factory pattern with Blueprints), SQLAlchemy ORM
Frontend	Jinja2 templates with Bootstrap 5 for a responsive UI
Database	SQLite for lightweight, file-based relational storage
Visualization	Chart.js for interactive analytics dashboards
Authentication	Flask-Login with custom role-based decorators; Werkzeug for password hashing
Migrations	Flask-Migrate for database schema migrations

2. System Architecture

Folder Structure

```
Project Root
├─ app/
│   ├─ _init_.py
│   ├─ models/
│   ├─ routes/
│   ├─ templates/
│   ├─ static/
│   ├─ utils/
│
├─ config.py
├─ run.py
├─ seed_admin.py
├─ migrate_db.py
└─ requirements.txt
```

Backend Architecture

The application is built using the Flask Application Factory pattern with four blueprints: Authentication, Admin, Staff, and Trekker. Each role has its own route file registered as a Flask Blueprint. Authentication is handled using Flask-Login with custom role-based decorators, and all endpoints enforce role-based access so that each user type can only reach their designated features and data.

Treks follow a controlled lifecycle, moving from Pending to Approved to Open, toggling between Open and Closed as needed, then to Ongoing, and finally Completed. A trek can only be booked while it is Open. When a trek is marked Completed, all active bookings automatically become Completed, populating each trekker's trekking history. All booking validations — duplicate prevention, slot checking, and status validation — are performed server-side.

```
Pending → Approved → Open ↔ Closed → Ongoing → Completed
```

Dashboards

Dashboards for Admin, Staff, and Trekker roles include Chart.js visualizations backed by SQL aggregation queries, giving each stakeholder a real-time view of the statistics relevant to their role.

3. Database Schema / ER Diagram

Tables

User — all application accounts.
Fields: id, username (unique), email (unique), password_hash, role, is_active, is_approved, created_at, updated_at.

Trek — stores trekking expedition information.
Fields: id, name, description, difficulty, duration_days, max_slots, available_slots, price, location, start_date, end_date, status, approved_at, completed_at, completion_notes, created_at, updated_at.

Booking — stores user bookings.
Fields: id, user_id, trek_id, booking_date, booking_status, payment_status, number_of_participants, total_amount, notes, completed_at, cancelled_at, created_at, updated_at.

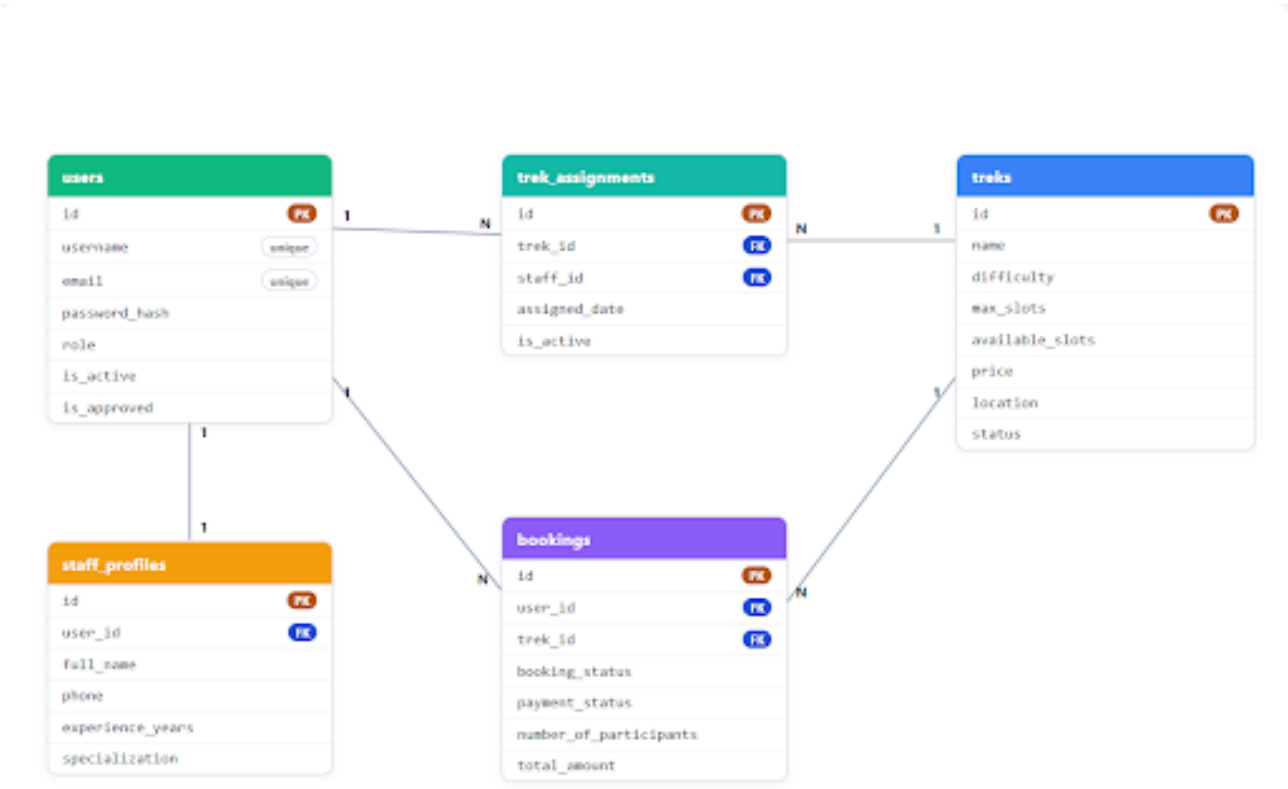
Staff Profile — stores staff details.
Fields: id, user_id, full_name, phone, experience_years, specialization, bio, created_at, updated_at.

Trek Assignment — stores trek assignments.
Fields: id, trek_id, staff_id, assigned_date, is_active.

Relationships

- User **1** → **N** Booking
- Trek **1** → **N** Booking
- User **1** → **1** Staff Profile
- Trek **N** ↔ **M** Staff (via Trek Assignment)

DB Diagram



The Trek Management application uses a relational database with **five main tables**:

- **Users** – Stores all user accounts (Admin, Trek Staff, and Trekker) along with authentication and role information.
- **Treks** – Stores trek details such as name, location, difficulty, available slots, price, and current status.
- **Bookings** – Records trek bookings made by trekkers and links users with treks.
- **Staff Profiles** – Stores additional profile information for trek staff.
- **Trek Assignments** – Maps trek staff to the treks assigned by the admin.

The relationships ensure that users can make multiple bookings, each trek can have multiple bookings, staff profiles are associated with staff accounts, and trek assignments allow administrators to assign staff members to specific treks.

4. Features Implemented

Authentication

Admin login, staff registration, and user registration are supported, with role-based access control enforced through Flask-Login integration.

Admin Dashboard

Admins get complete platform oversight: a dashboard, full trek CRUD, staff approval and staff assignment, user and staff blacklisting, search, booking management, and access to trek history.

Trek Staff Dashboard

Trek Staff can view their assigned treks, update available slots and trek status, manage participants, and add completion notes once a trek concludes.

Trekker Dashboard

Trekkers can browse and search treks with filtering, book a trek, cancel a booking, view their booking status, review their trekking history, and edit their profile.

Booking Rules

- Prevent overbooking
- Prevent duplicate booking
- Booking only allowed while a trek is Open
- Slot validation on every booking request
- Automatic history update when a trek is completed

Analytics

Admin: total treks, users, staff, bookings, revenue, and trek status breakdown.
Staff: participants, booking status, and assigned treks.
Trekker: trek history, monthly trend, and total spending.

Additional Features

- Trek lifecycle management
- WCAG accessibility improvements
- Responsive Bootstrap layout
- Reusable Chart.js components

- Server-side validation
- Secure password hashing
- Role-based route protection

5. API Resource Endpoints

Endpoint	Method	Description
/admin/api/analytics	GET	Admin analytics dashboard
/staff/api/analytics	GET	Staff analytics dashboard
/trekker/api/analytics	GET	Trekker analytics dashboard

Features

- Authentication required
- Role-based authorization
- SQL aggregation using GROUP BY
- JSON responses
- 403 returned for unauthorized users

6. AI Usage Declaration

Percentage of AI Used: 5% (Estimated)

I used AI through Gemini during the development of this project, mainly for a small amount of assistance while building the Chart.js visualizations on the dashboards. AI support was limited to helping resolve a specific issue faced while wiring up the Chart.js charts to the dashboard data.

All project requirements, architecture decisions, database schema design, API implementation, booking-integrity logic, testing, and the final submission were designed, implemented, and reviewed manually by me. Every AI-assisted snippet was reviewed, understood, and tested before integration.

7. Demo Video

A recorded demonstration of the Trek Management App is available for review. The video walkthrough covers the complete application workflow, including trek publishing and approval, staff assignment, trek booking, booking-integrity rules in action, and the trek completion flow.

Video Link: *MAD-1 Trekk Management video.mp4*